

単一編集者の推敲技能を学習する評価器の設計

編集者の技能を条件付き分布として表現できるか

実験名 editor-as-distribution / 想定読者: PRML 読了程度 / 実行環境: VRAM 24 GB 単体 GPU

1 問題の設定

1.1 主仮説

この実験が確かめること

この編集者にとっての「良い推敲」は、質の軸上の高さではなく、この編集者の条件付き分布への近さとして測れる。

既存の選好モデル群は「質を順序として表現する」試みであった。本実験はその否定側に立った上で、では何として表現できるのかに答えを出す。答えの候補が分布である。下位命題は三つ。

H1 密度（対数尤度比）が、順序を学習したモデルを**難しい境界**で上回る。

H2 比較不可能な候補は**欠陥**を持つ。関門で落ちる件数が有意にあり、関門を外すと性能が落ちる。

H3 密度だけでは**負の推敲を検出できない**。絶対ラベルが要る。

H3 が、関門と順位づけを分ける本当の理由である。密度は「近いか遠いか」しか言わず、**遠さの方向を持たない**。 s_{den} が低い候補が、下書きより劣化しているのか、単にこの編集者らしくない別の良い推敲なのかを、密度は区別できない。この区別には原点と、原点の下方の水準が要る。

1.2 目標

ある一人のプロ編集者の推敲を評価する関数を得る。編集者が一人であり、かつ個人語法そのものが技能であるという前提により、技能と個人語法を分離する識別問題は生じない。したがって学習対象は次の条件付き分布そのものでよい。

$$p_{\text{ed}}(y \mid x), \quad x = \text{下書き}, \quad y = \text{推敲文}.$$

下書きを基準とした差を

$$\Delta(x, y) = s(x, y) - s(x, x) \quad (1)$$

と書き、 $\Delta > 0$ を「この編集者の推敲に近い」、 $\Delta \leq 0$ を「遠い」と解釈する。

ここで推敲 (revision) は局所的な語句置換の集合ではなく、大局的な文意を強く保持しながら局所的な意味の変更を含む文章全体の書き換えである。この定義の下では、変更前スパンと変更後スパンの対応づけを前提とする手法は使えない。段落の統合、論の順序の再配置、主題の繰り上げに対してスパン対応が定義できないためである。本設計は**編集アラインメントを一切要求しない**。

1.3 比較不可能性は次元ではなく欠陥に由来する

「潜在的な推敲の質がスカラーとして埋め込めるか」は検証すべき仮説であり、おそらく成り立たない。その理由は重要である。候補どうしがしばしば**比較不可能**だからであり、そしてその比較不可能性は次のような事象から生じる。

- 一方に致命的な欠落がある
- 一方に致命的な書き足しがある
- 一方が実質的に無修正である

これらは「質の次元が複数ある」という話ではない。いずれも欠陥である。欠陥を含む候補は、質の軸上のどこかに位置づけられるのではなく、そもそも軸に乗らない。

設計上の中心的帰結

比較不可能性が欠陥に由来するならば、対処は「軸を増やす」ことではなく「軸に乗せる前に落とす」ことである。すなわち評価器は単一の得点関数ではなく、**関門 (gate) の列と、その後段の順位づけ (ranker)** として構成されなければならない。

これは多次元の選好モデルや歪対称な一般選好モデルによっては解決しない。それらが緩和するのは非推移性であって、欠陥による定義域外への逸脱ではない。

この観点は、以降のデータ配分の判断をすべて決定する。関門と順位づけは異なる判別問題であり、したがって異なるデータで学習されるべきである。

2 選好モデルが破綻する三つの構造的理由

観測されている症状は三つある。負の推敲を評価できないこと、文字数の差を見ていること、良し悪しを線形にしか評価できないこと。これらは独立した課題ではなく、「文書全体に対する単一の潜在スカラーを学習する」という設計から導かれる同一の帰結である。損失を Bradley-Terry から一般選好モデルへ、あるいは P-DPO へ取り替えても解けない。

2.1 平行移動不変性 — 絶対原点が存在しない

Bradley-Terry モデルは各対象に潜在的な強さ $f(y) \in \mathbb{R}$ を割り当て、比較を

$$P(y_1 \succ y_2) = (f(y_1) - f(y_2)) \quad (2)$$

で表す。

PRML 4.3: ロジスティック回帰。式 (2) は差分特徴に対するロジスティック回帰と同型である。

f と $f + c$ は任意の定数 c について全く同じ選好分布を与える。ゆえに「**絶対的にどの水準か**」を表す目盛りが存在しない。式 (1) のように下書きを基準に据えれば原点は導入できるが、それは $s(x, x)$ という一点を原点に固定するだけであり、**目盛りの刻み**を与えない。「下書きよりどれだけ悪いか」を表現するには、原点の下方に少なくとも一つの学習された水準が必要である。

2.2 生成過程の交絡 — 表層統計量への崩落

症状の第二は、しばしばモデルの怠慢と解釈されるが、そうではない。モデルは正しく最適化している。

人間の推敲を正例、それ以外を負例とする判別問題において、ベイズ決定理論の観点では最適な決定関数はクラス条件密度の比に依存する。

PRML 1.5: 決定理論。4.2: 確率的生成モデルと、事後確率がロジスティックシグモイドの形をとること。

問題は、正例と負例が**質以前に異なる生成過程から来ている**点である。編集量、長さの変化、語彙分布、文長の分散が系統的に異なる。この周辺特徴がすでに二群を分離しているならば、尤度比はその特徴でほぼ決まる。質という高次の特徴を抽出するより、文字数という一次元の統計量を読む方が経験リスクを下げるコストが圧倒的に低い。

すなわち**データが提示していた十分統計量が文字数だった**のであり、これは損失の問題ではなく負例集団の構成の問題である。

退化の兆候

下書きを人間の推敲より上位に置いた件数が検証集合で厳密に 0 になる場合、それは性能が高いのではなく「変更されている ⇒ 良い」という規則に完全に崩落している兆候である可能性が高い。誤り率が低すぎる場合は、まずショートカットを疑う。

2.3 全順序の仮定 — 多峰性を潰す

f が実数値である以上、任意の二候補に必ず優劣がつく。しかし推敲には「等しく優れているが互いに全く異なる」複数の解が存在する。これが症状の第三である。前節で述べたとおり、欠陥に由来する比較不可能性はこれとは別の問題であり、両者を混同してはならない。

3 設計原則: 関門と順位づけの分離

以上より、評価器を次の構成に分解する。

段	構成要素	判定内容
G1	欠陥関門	致命的欠落・致命的書き足し・実質無修正の検出。軸に乗るか否かの判定であり、質の判定ではない
G2	水準関門	下書き水準以下か否かの判定。絶対原点の下方を表現する
R	順位づけ	関門を通過した候補の中での、この編集者らしさ。密度であり順序ではない
A	補助信号	大局性。段落をまたぐ推敲に限り有効
C	回送	確信度による人手への escalation

データ配分の原則

関門と順位づけは異なる判別問題である。したがって同じデータで学習してはならない。

- 関門が解くのは「質の軸に乗るか」「原点より下か」という粗い判別である。易しい負例で学習できるし、易しい負例でしか学習できない。欠陥は定義上、明瞭だからである。
- 順位づけが解くのは、いずれも欠陥がなく劣化もしていない候補どうしの細かい判別である。ここには難しい負例が必要である。

この区別を誤ると、粗い判別で学習した表現を細かい判別に転用することになり、後述する転移の非対称性によって性能が崩れる。

4 関門 G2: 絶対原点を持つ累積リンクモデル

欠陥関門 G1 はほぼ規則で実装できるため詳述しない（文長比の極端な逸脱、内容語の欠落率、正規化編集距離のゼロ近傍）。ここでは G2 を扱う。

4.1 累積リンクモデル

「下書き水準より上か下か」を学習するには、原点を含みかつ原点の下方に水準を持つ絶対ラベルが要る。相対比較ラベルからは原理的に構成できない。

順序カテゴリを多クラス分類として扱えば順序情報が失われ、回帰として扱えばカテゴリ間隔の等間隔性を仮定してしまう。適切な定式化は潜在変数の閾値モデルである。潜在的な質を

$$z = g(x, y) + \varepsilon, \quad \varepsilon \sim \text{Logistic}(0, 1) \quad (3)$$

と置き、順序つき閾値 $\theta_1 < \theta_2 < \theta_3$ により観測カテゴリが定まるとする。累積確率は

$$P(Y \leq k | x, y) = (\theta_k - g(x, y)), \quad k = 1, 2, 3 \quad (4)$$

となり、カテゴリ確率は差分 $P(Y = k) = (\theta_k - g) - (\theta_{k-1} - g)$ で得られる。

PRML 4.3.2: ロジスティック回帰と潜在変数表現。式 (4) はその順序多値版であり、リンク関数をプロビットに替えれば PRML 4.3.5 の probit 回帰に対応する。閾値の単調性は $\theta_1 = b_1$, $\theta_k = \theta_{k-1} + \text{softplus}(b_k)$ と再パラメータ化すれば自動的に満たされる。

下書き水準に対応する閾値を θ_2 とすれば、関門の判定は

$$P(\text{下書きより上}) = 1 - (\theta_2 - g(x, y)) \quad (5)$$

として直接読める。 θ_2 がデータから推定された絶対原点として機能している。式 (2) にはこれに相当するものが存在せず、ゆえに同じ問いを立てることができなかった。

4.2 損失と評価指標

学習には式 (4) から導かれるカテゴリ確率の負の対数尤度を用いる。対数スコアは proper scoring rule であり、その期待値を最小化する点が真の条件付き確率に一致する。すなわち**較正は損失関数の性質として自動的に得られる**。強化学習も事後的な較正手法も不要である。

評価には Ranked Probability Score を用いる。

$$\text{RPS} = \frac{1}{K-1} \sum_{k=1}^{K-1} (F_k^{\text{pred}} - F_k^{\text{obs}})^2, \quad F_k = \sum_{j \leq k} P(Y = j). \quad (6)$$

RPS は順序カテゴリに対する proper scoring rule であり、**隣接カテゴリの誤りと遠方カテゴリの誤りを区別する**。通常の Brier score や正解率は順序を無視するため、この設定では不適切である。

4.3 絶対ラベルデータを G2 に限定して使う理由

絶対ラベル付きのデータ（下書き水準を含む多段階ラベル）は、原点の下方を表現できる唯一の資源である。しかしそれを順位づけにも用いることは**推奨されない**。理由は次節の転移の非対称性にある。

4.4 転移の非対称性 — 易しい負例は難しい境界を教えない

絶対ラベル付きデータの負例は、通常、汎用の言語モデルに下書きを推敲させて作る。一方、実運用で順位づけが直面する候補は、この編集者のデータで適応させた生成器の出力である。両者は難易度が根本的に異なる。

二つの判別問題

易しい境界: 編集者 vs 汎用モデル。表層と語彙分布が大きく違うため、粗い特徴で分離できる。

難しい境界: 編集者 vs 編集者を模倣する適応済み生成器。表層が近いため、個人語法の細部でしか分離できない。

易しい境界で学習した表現は、難しい境界に転移しない。汎用モデルと編集者を分ける特徴が、模倣器と編集者を分ける特徴と一致する保証がないからである。実際、絶対ラベルデータの内部では高い一致率が得られるのに、適応済み生成器の出力を含む検証集合では偶然水準をわずかに上回る程度にとどまる、という乖離が起きうる。この乖離が観測された場合、それは学習の失敗ではなく**境界の取り違い**である。

さらに二つの交絡が重なりうる。

- **単位の不一致。** 絶対ラベルデータが段落内の推敲から構成され、目標タスクが段落をまたぐ推敲を主とするならば、判断の単位そのものが異なる。
- **目盛りの密度の不一致。** 絶対ラベルが原点の下方に厚く、原点の上方（「良い」と「優れている」の間）に薄いならば、目標タスクが要求する解像度がそこに存在しない。

必須の診断

絶対ラベルデータを使う前に、**原点より上の水準どうしのペアだけに制限した一致率**を測ること。全体の一致率が高くてこの部分集合で偶然水準に落ちるなら、そのデータは関門にしか使えず、順位づけには使えない。

これは一度の集計で済み、以降のデータ配分の判断をすべて決める。

5 順位づけ R: 密度としての個人語法

5.1 対数尤度比

ベース言語モデルを p_0 、これに低ランク適応を付与して編集者の推敲ペアで学習したものを p_θ とする。順位づけのスコアを

$$s_{\text{den}}(x, y) = \log p_\theta(y | x) - \log p_0(y | x) \quad (7)$$

と定義する。これは対数尤度比であり、「この推敲は編集者の分布から生成されたか、汎用モデルの分布から生成されたか」という単純仮説検定において、Neyman–Pearson の補題により**最強力検定の検定統計量**となる。

実務的には、分母 p_0 が「日本語として自然であること」「長いこと」「一般的な語彙を使っていること」といった編集者に固有でない寄与を一次の近似で相殺する。

p_0 は p_θ から適応を外したものであるから、同一の重みを共有する。参照モデルを別途 VRAM に載せる必要がなく、適応の on/off を切り替えて二回 forward するだけでよい。24 GB という制約下でこれは決定的である。

5.2 多峰性

式 (7) は順序ではなく**密度**である。密度は混合を許す。ある下書きに対して妥当な推敲が三通りあり、編集者がそれらを 0.5/0.3/0.2 の割合で選ぶならば、 p_θ はそのまま三峰の分布として表現できる。三つすべてに高い値を与えられるので、優劣をつける必要がない。式 (2) の全順序の仮定がここにはない。

5.3 難しい負例をどこから取るか

順位づけの学習と検証には、**欠陥がなく、かつ下書きより劣化していないことが確認された対抗候補**が必要である。この条件を満たす負例こそが難しい境界を定義する。

重要なのは、こうした対抗候補を**学習に使うか検証に使うか**の判断である。式 (7) の学習自体は編集者の推敲のみを用いるため負例を必要としない。したがって、劣化していないことが確認済みの対抗候補は、**検証側に温存する**のが合理的である。数が限られている資源を、交絡の入りうる学習側ではなく、難しい境界の測定に充てる。

5.4 長さの効果

比を取っても長さの効果は完全には消えない。 p_θ と p_0 の長さ依存が厳密には一致しないためである。三通りの正規化（総和、トークン平均、長さ文字数差への回帰残差）を実装し、**層化した検証性能に基づいて選択**する。この選択を暗黙に決めてはならない。

6 補助信号 A: 大局性

熟練した推敲では、局所的な変更が大局的な再把握に駆動されている。この場合、離れた箇所の変更どうしは統計的に従属する。表層的な推敲では各箇所の変更が独立に決まる。

推敲文 y を K 個のブロック $y_1, \dots, y_K$ に分割し、学習していない素の言語モデル M を用いて

$$G(x, y) = \frac{1}{K} \sum_{k=1}^K \frac{1}{|y_k|} \left[\log p_M(y_k | x, y_{-k}) - \log p_M(y_k | x, \tilde{y}_{-k}) \right] \quad (8)$$

を定義する。 $\tilde{y}_{-k}$ は別の文書の推敲ブロックであり、 y_{-k} とトークン長を整合させたものである。式 (8) は条件付き相互情報量 $I(y_k; y_{-k} | x)$ のモンテカルロ推定である。

PRML 1.6: エントロピーと相互情報量。 $I(a; b) = H(a) - H(a | b)$ は、 b を知ることで a の不確実性がどれだけ減るかを表す。

統制条件の必要性

素朴には「文脈 = x のみ」との差を取りたくなるが、それは誤りである。 y_{-k} を加えた条件は文脈が長い
ため、内容と無関係に対数尤度がほぼ必ず上昇する。その差を報告することは**文脈長効果を大局性と
呼んでいるにすぎない**。

$\tilde{y}_{-k}$ は同じだけの文脈長を持ちながら当該文書の推敲に関する情報を含まない。ゆえに式 (8) の差だけが
「この文書が実際にどう推敲されたか」の情報を取り出す。

式 (8) は y を分割するだけで、 x との対応づけを行わない。段落の統合も順序の入れ替えも主題の繰り上げも、条件づけの対象に入るだけであり定義上の問題を起こさない。また編集を型に分類する必要もない。
macrostructure に駆動されているか否かを、変更どうしの統計的従属性として直接測っている。

適用範囲は段落をまたぐ推敲に限る。段落内の推敲には定義できない。

7 評価設計

7.1 床と天井

すべての主張は二つの基準の間に位置づけて述べる。

床。表層特徴のみ（文字数差、正規化編集距離、文長の平均と分散、type-token ratio、句読点密度、漢字含有率）を入力とする順序ロジスティック回帰。既存モデルの性能がこの床と同程度なら、それらが学習していたのは表層のみだったと確定できる。定性的な「決め手に欠ける」状態を定量的な診断に変えるのがこの工程の目的である。

天井。本設計における最終的な比較対象は、推敲を行った本人によるブラインド判定である。ここで決定的な事実として、**本人自身の自己一致率が 100% ではない**。適応済み生成器の出力は、しばしば本人にも区別がつかない。したがって

$$\text{達成可能な一致率の上限} = \text{本人のブラインド自己一致率} < 1.$$

100% を目標に据えることは要件の誤設定であり、そこへ向けた最適化はむしろ有害である。報告は常に「床からの上昇分／天井までの残り」として行う。

7.2 指標の統一

次の二つは異なる量であり、混同してはならない。

量	定義と注意
勝率	評価器が編集者の推敲を対抗候補より上位に置いた割合。天井が 1 でないため、これが高いほど良いとは限らない
一致率	評価器の選好の符号が、本人のブラインド判定と一致した割合。 これが主指標である

比較可能性の要件

複数のモデルを比較するとき、**全モデルが同一の検証集合上で同一の指標により評価されていないと**ならない。あるモデルを勝率で、別のモデルを一致率で報告した表は、比較の体をなしていない。過去の実験が異なる指標で記録されている場合は、再計算してから比較すること。

7.3 層化

すべての評価を、下書きからの文字数差の十分位で層化して報告する。層内で性能が偶然水準に落ちるなら、全体性能は文字数由来だったことになる。これは順位相関よりはるかに診断的である。

また、順序との相関（Spearman 等）を主指標にしてはならない。順序が存在しないという前提と整合しないためである。

8 リークと交絡の管理

最も見落とされやすい罠

絶対ラベル付きデータの下書きが、密度モデルの学習コーパスの部分集合である場合、そのまま学習したモデルで当該データを評価すると、編集者の推敲は学習済みである。閾門の評価は無効になり、順位づけのスコアも楽観的に偏る。

対策として、**当該下書きを除外した適応と、全量の適応の二つを作る**。除外版を閾門と難負例検証に、全量版を最終検証に用いる。除外による性能差自体も記録する。

その他、着手前に全数で確認すべき事項:

- 各検証取り分けが、対応する学習分割に混入していないこと
- 最終検証集合の全項目が取り分け由来であること
- 相対比較データの検証分割が、下書きの単位で学習分割と分離されていること（同一下書きから作られた複数のペアが学習と検証に分かれてはならない）
- 交差検証が下書き単位のグループ分割であること

9 反証条件と実行順序

各段階に明示的な go / no-go を置く。

段階	支持条件	反証された場合
床の確定	—	診断のみ。既存モデルとの比較基準を得る
目盛りの診断	原点より上の水準どうしに制限した一致率が偶然水準を明確に上回る	絶対ラベルデータは G2 専用とし、順位づけからは完全に外す
大局性	群間差が有意で、文字数差との偏相関が小さく、 K に対して符号が安定	仮説を棄却し、補助信号を設計から除去する。否定的結果として記録する
順位づけ	難負例検証における一致率が、文字数層内で床を有意に上回る	適応の rank と学習率を探索し、届かなければデータ量不足として報告
関門	原点下方の水準の再現率が層内で床を上回る	絶対ラベルの追加収集が最優先の投資先だと判明する

実行順序

大局性の検証は学習を伴わないため最初に決着させる。棄却された場合、以降の設計から補助信号を丸ごと除去でき、数週間が節約できる。

目盛りの診断は一度の集計で済み、絶対ラベルデータを関門専用にするか順位づけにも使うかというデータ配分の根本判断を決める。順位づけの学習より前に必ず行う。

反証条件を後から緩めてはならない。棄却されたときに指標を作り直すのは、この計画全体の意味を壊す。

9.1 主仮説が偽になる条件

H1 が偽なら主仮説が棄却され、「順序でも分布でもない第三の表現が要る」という結論になる。H2 と H3 が偽なら、関門と順位づけの分離が不要だったことになり、設計は単一の密度モデルに縮退する。

#	偽になる条件
H1	密度モデルの難負例検証における一致率が、既存の順序モデルの最良値を文字数層内で上回らない
H2	欠陥関門がほとんど何も落とさない、または関門を外した ablation で最終性能が変わらない
H3	水準関門の判定が、順位づけスコアの単調変換で代替できる

実験名がそのまま検証対象である。

9.2 この設計が答えないこと

誠実のために明記する。

- **生成能力そのものは向上しない。**本設計は評価器の構築であり、生成器を改善するものではない。ただし複数候補から選ぶ工程を改善することで、実効的な出力品質は向上する。
- **他者・他ジャンルへの汎化は保証されない。**学習されるのはこの編集者の分布である。ただし本件の目的はまさにこの編集者の技能の抽象化であるから、これは制約ではなく仕様である。
- **判断の根拠は言語化されない。**密度モデルも累積リンクモデルも、なぜその推敲が良いのかを説明しない。説明可能性が要件に含まれるなら別の設計が必要である。

- 天井は動かせない。当人のブラインド自己一致率を超える一致率は、定義上、測定できない。